Context-Free Session Types with Borrowing: Supplementary Material

ANONYMOUS AUTHOR(S)

The organization of the supplement follows the structure of the main document. Topics are covered in the same sequence, so the two documents can be read side-by-side.

S-A Motivation

S-A.1 Remarks

The reader may ask why **select** still adheres to resource-passing style. The answer is simplicity of our system. While it is possible to define **select** with a borrowing-based interface, supporting it requires subtyping of session types. While subtyping is fine in theory, it turns out that it is undecidable for CFST [Padovani 2019]. Hence, it cannot be the basis for a complete type checker.

For borrowing from regular session types [Saffrich et al. 2025], a borrowing-style **select** operation is possible because subtyping for regular session types is decidable.

A borrowing-style **branch** operator (external choice) cannot be supported in CSTB. This operator selects between different continuations at run time and the residual channel depends on this run-time choice.

All CSTB examples are available as part of our supplement. Compared to the presentation in the main text, the split operators carry type parameters in the actual code to simplify type checking.

$$\textsc{U-Unit}\ \ \mathsf{unr}\ \mathtt{Unit} \qquad \textsc{U-Arr}\ \ \mathsf{unr}\ (T_1 \to^{\mathrm{M}}_{\omega,\epsilon} T_2) \qquad \textsc{U-Prod}\ \ \frac{\mathsf{unr}\ T_1 \qquad \mathsf{unr}\ T_2}{\mathsf{unr}\ (T_1 \otimes_d T_2)} \qquad \textsc{U-Variant}\ \ \frac{\forall \ell.\ \mathsf{unr}\ T_\ell}{\mathsf{unr}\ \langle \ell : T_\ell \rangle_{\ell \in L}}$$

$$\textsc{U-CtxEmpty}\ \ \mathsf{unr}\ \emptyset \qquad \textsc{U-CtxVar}\ \ \frac{\mathsf{unr}\ T}{\mathsf{unr}\ x : T} \qquad \textsc{U-CtxSeq}\ \ \frac{\mathsf{unr}\ \Gamma_1 \qquad \mathsf{unr}\ \Gamma_2}{\mathsf{unr}\ (\Gamma_1, \Gamma_2)} \qquad \textsc{U-CtxPar}\ \ \frac{\mathsf{unr}\ \Gamma_1 \qquad \mathsf{unr}\ \Gamma_2}{\mathsf{unr}\ (\Gamma_1 \parallel \Gamma_2)}$$

$$\textsc{TN-Skip}\ \ \mathsf{new}\ \mathtt{Skip} \qquad \textsc{TN-Send}\ \ \mathsf{new}\ !T \qquad \textsc{TN-Recv}\ \ \mathsf{new}\ ?T \qquad \textsc{TN-Var}\ \ \mathsf{new}\ X$$

$$\textsc{TN-Seq}\ \ \frac{\mathsf{new}\ S_1 \qquad \mathsf{new}\ S_2}{\mathsf{new}\ (S_1; S_2)} \qquad \textsc{TN-Branch}\ \ \frac{\forall \ell.\ \mathsf{new}\ S_\ell}{\mathsf{new}\ \&\{\ell : S_\ell\}_{\ell \in L}} \qquad \textsc{TN-Choice}\ \ \frac{\forall \ell.\ \mathsf{new}\ S_\ell}{\mathsf{new}\ \oplus\{\ell : S_\ell\}_{\ell \in L}} \qquad \textsc{TN-Rec}\ \ \frac{\mathsf{new}\ S}{\mathsf{new}\ \mu X.S}$$

Fig. S-1. Unrestricted types and contexts ($\mathsf{unr}\ T$, $\mathsf{unr}\ \Gamma$); session types valid for new ($\mathsf{new}\ S$)

Laws for context-free session types.

$$\mathtt{Skip}; S = S \qquad S; \mathtt{Skip} = S \qquad S_1; (S_2; S_3) = (S_1; S_2); S_3 \qquad \mu X.S = S[\mu X.S/X]$$

$$(\oplus\{\ell : S_\ell\}_{\ell \in L}); S = \oplus\{\ell : S_\ell; S\}_{\ell \in L} \qquad (\&\{\ell : S_\ell\}_{\ell \in L}); S = \&\{\ell : S_\ell; S\}_{\ell \in L}$$

Laws for duality $S^\perp$ (there is no dual for Drop and Acq).

$$\mathtt{Skip}^\perp = \mathtt{Skip} \qquad (S_1; S_2)^\perp = {S_1}^\perp; {S_2}^\perp \qquad !T^\perp = ?T \qquad ?T^\perp = !T$$

$$(\oplus\{\ell : S_\ell\}_{\ell \in L})^\perp = \&\{\ell : {S_\ell}^\perp\}_{\ell \in L} \qquad (\&\{\ell : S_\ell\}_{\ell \in L})^\perp = \oplus\{\ell : {S_\ell}^\perp\}_{\ell \in L} \qquad \mathtt{Close}^\perp = \mathtt{Wait}$$

$$\mathtt{Wait}^\perp = \mathtt{Close} \qquad (\mu X.S)^\perp = S[\mu X.S/X]^\perp$$

Fig. S-2. Session-type equality and duality

S-B Types: Session-Type Laws and Well Formedness Predicates

This section contains figs. S-1 and S-2.

$$
\begin{array}{rcll}
u, v & ::= & c \mid x \mid \lambda x.e \mid \mathtt{inj}_i\, v \mid u \otimes v & \text{(Values)} \\
\mathcal{E} & ::= & \Box \mid (\mathcal{E} \mathbin{@_{\mathrm{L}}} e) \mid (v \mathbin{@_{\mathrm{L}}} \mathcal{E}) \mid (e \mathbin{@_{\mathrm{R}}} \mathcal{E}) \mid (\mathcal{E} \mathbin{@_{\mathrm{R}}} v) \mid \mathcal{E}; e \mid \mathcal{E} \otimes e \mid v \otimes \mathcal{E} & \\
& & \mid \mathtt{inj}_i\, \mathcal{E} \mid \mathtt{let}\ x \otimes x = \mathcal{E}\ \mathtt{in}\ e \mid \mathtt{case}\, \mathcal{E}\, \{\overline{\mathtt{inj}_i\, x \Rightarrow e}\} & \text{(Evaluation contexts)} \\
\mathcal{F} & ::= & \langle \mathcal{E} \rangle & \text{(Process evaluation contexts)} \\
\mathcal{G} & ::= & \Box \mid \mathcal{G} \parallel P \mid (\nu[B][B])\mathcal{G} & \text{(Process contexts)}
\end{array}
$$

Fig. S-3. Values and evaluation contexts

$$
\textsc{E-App}\ (\lambda x.e)\, v \longrightarrow_e e[v/x] \qquad \textsc{E-Seq}\ v; e \longrightarrow_e e \qquad \textsc{E-PairElim}\ \mathtt{let}\ x \otimes y = u \otimes v\ \mathtt{in}\ e \longrightarrow_e e[v/x][u/y]
$$

$$
\textsc{E-Unfold}\ \mu f.e \longrightarrow_e e[\mu f.e/f] \qquad \textsc{E-VariantElim}\ \frac{k \in L}{\mathtt{case}\,(\mathtt{inj}_k\, v)\,\{\mathtt{inj}_\ell\, x_\ell \Rightarrow e_\ell\}_{\ell \in L} \longrightarrow_e e_k[v/x_k]} \qquad \textsc{E-Ctx}\ \frac{e_1 \longrightarrow_e e_2}{\mathcal{E}[e_1] \longrightarrow_e \mathcal{E}[e_2]}
$$

Fig. S-4. Expression reduction ($e \longrightarrow_e e$)

$$
\textsc{C-ParComm}\ P_1 \parallel P_2 \equiv P_2 \parallel P_1 \qquad \textsc{C-ParAssoc}\ P_1 \parallel (P_2 \parallel P_3) \equiv (P_1 \parallel P_2) \parallel P_3 \qquad \textsc{C-ParUnit}\ \langle * \rangle \parallel P \equiv P
$$

$$
\textsc{C-ResSwap}\ (\nu[B_1][B_2])P \equiv (\nu[B_2][B_1])P \qquad \textsc{C-ResComm}\ \frac{(\mathsf{fv}(B_1) \cup \mathsf{fv}(B_2)) \cap (\mathsf{fv}(B_3) \cup \mathsf{fv}(B_4)) = \emptyset}{(\nu[B_1][B_2])(\nu[B_3][B_4])P \equiv (\nu[B_3][B_4])(\nu[B_1][B_2])P}
$$

$$
\textsc{C-Extend}\ \frac{\mathsf{fv}(P_1) \cap (\mathsf{fv}(B_1) \cup \mathsf{fv}(B_2)) = \emptyset}{P_1 \parallel (\nu[B_1][B_2])P_2 \equiv (\nu[B_1][B_2])(P_1 \parallel P_2)}
$$

Fig. S-5. Process congruence

S-C Dynamics: Congruence and Reduction Rules

This section contains figs. S-3 to S-5.

TrC-ParComm
$$P_1 \parallel P_2 \equiv P_2 \parallel P_1$$

TrC-ParAssoc
$$P_1 \parallel (P_2 \parallel P_3) \equiv (P_1 \parallel P_2) \parallel P_3$$

TrC-ParUnit
$$\langle * \rangle \parallel P \equiv P$$

TrC-ResSwap
$$(\nu xy)P \equiv (\nu yx)P$$

TrC-ResComm
$$\frac{\{x_1, y_1\} \cap \{x_2, y_2\} = \emptyset}{(\nu x_1 y_1)(\nu x_2 y_2)P \equiv (\nu x_2 y_2)(\nu x_1 y_1)P}$$

TrC-FlagComm
$$\frac{z_1 \neq z_2}{(\varphi z_1 \mapsto \phi_1)(\varphi z_2 \mapsto \phi_2)P \equiv (\varphi z_2 \mapsto \phi_2)(\varphi z_1 \mapsto \phi_1)P}$$

TrC-ResFlagComm
$$\frac{z \notin \{x, y\}}{(\nu xy)(\varphi z \mapsto \phi)P \equiv (\varphi z \mapsto \phi)(\nu xy)P}$$

TrC-ResExtend
$$\frac{x, y \notin \mathsf{fv}(P_1)}{P_1 \parallel (\nu xy)P_2 \equiv (\nu xy)(P_1 \parallel P_2)}$$

TrC-FlagExtend
$$\frac{z \notin \mathsf{fv}(P_1)}{P_1 \parallel (\varphi z \mapsto \phi)P_2 \equiv (\varphi z \mapsto \phi)(P_1 \parallel P_2)}$$

Fig. S-6. Congruence rules of translated processes

S-D Translation: Congruence of Translated Processes

This section contains fig. S-6.

$$\Gamma_1 \overset{d}{\bowtie} \Gamma_2 = \begin{cases} \Gamma_1 \parallel \Gamma_2 & \text{if } d \in \{1, \omega\} \\ \Gamma_1, \Gamma_2 & \text{if } d = \text{L} \\ \Gamma_2, \Gamma_1 & \text{if } d = \text{R} \end{cases} \qquad \Gamma|_X = \begin{cases} \Gamma_1|_X \parallel \Gamma_2|_X & \text{if } \Gamma = \Gamma_1 \parallel \Gamma_2 \\ \Gamma_1|_X, \Gamma_2|_X & \text{if } \Gamma = \Gamma_1, \Gamma_2 \\ x : T & \text{if } \Gamma = x : T \text{ and } x \in X \\ \emptyset & \text{otherwise} \end{cases}$$

$$\text{joinDir}(\Gamma, X) = \begin{cases} \mathbf{1} & \text{if } \Gamma|_X \parallel \Gamma|_{X^C} \preccurlyeq \Gamma \\ \text{L} & \text{if } \Gamma|_X, \Gamma|_{X^C} \preccurlyeq \Gamma \\ \text{undef.} & \text{otherwise} \end{cases}$$

Fig. S-7. Context join $\Gamma_1 \overset{d}{\bowtie} \Gamma_2$ joins two contexts according to the direction d, context restriction $\Gamma|_X$ restricts a context to a set of variables X, and joinDir calculates if a context Γ restricted to a set X can be joined parallelly with Γ restricted to the complement set.

CS-Pair
$$T^* \otimes_d U^* \simeq T^{*\prime} \otimes_d U^{*\prime} \Longrightarrow \{T^* \simeq T^{*\prime}, U^* \simeq U^{*\prime}\}$$

CS-Sum
$$T^* + U^* \simeq T^{*\prime} + U^{*\prime} \Longrightarrow \{T^* \simeq T^{*\prime}, U^* \simeq U^{*\prime}\}$$

CS-Unit
$$\texttt{Unit} \simeq \texttt{Unit} \Longrightarrow \emptyset$$

CS-Arrow
$$T^* \to^m_{d,\epsilon} U^* \simeq T^{*\prime} \to^m_{d,\epsilon} U^{*\prime} \Longrightarrow \{T^{*\prime} \simeq T^*, U^* \simeq U^{*\prime}\}$$

CS-Swap
$$\frac{T^* \notin A}{T^* \simeq \alpha \Longrightarrow \{\alpha \simeq T^*\}}$$

CS-Context
$$\frac{T^* \simeq U^* \Longrightarrow \Delta'}{\Delta \cup \{T^* \simeq U^*\} \Longrightarrow \Delta \cup \Delta'}$$

Fig. S-8. Constraint simplification

S-E Algorithmic Typing

This section contains figs. S-7 and S-8.

$\boxed{T_{in} \mathbin{+\!\!+} T_{in} = T_{out}}$

$$\textsc{TC-Skip}\ \frac{}{T \mathbin{+\!\!+} \texttt{Skip} = T} \qquad \textsc{TC-SemiDist}\ \frac{V \neq \texttt{Skip}}{(T;U) \mathbin{+\!\!+} V = T;(U \mathbin{+\!\!+} V)} \qquad \textsc{TC-NonSemi}\ \frac{T \neq V;W \qquad U \neq \texttt{Skip}}{T \mathbin{+\!\!+} U = T;U}$$

$\boxed{T_{in} \Downarrow T_{out}}$

$$\textsc{TN-SemiSkips}\ \frac{T \simeq \texttt{Skip} \qquad U \Downarrow U'}{T;U \Downarrow U'} \qquad \textsc{TN-Semi}\ \frac{\neg(T \simeq \texttt{Skip}) \qquad T \Downarrow T'}{T;U \Downarrow T' \mathbin{+\!\!+} U} \qquad \textsc{TN-Mu}\ \frac{S \Downarrow S'}{\mu X.S \Downarrow S'[\mu X.S'/X]}$$

$$\textsc{TN-ChoiceSemi}\ \frac{}{\oplus\{\ell : S_\ell\}_{\ell \in L};T \Downarrow \oplus\{\ell : S_\ell;T\}_{\ell \in L}} \qquad \textsc{TN-BranchSemi}\ \frac{}{\&\{\ell : S_\ell\}_{\ell \in L};T \Downarrow \&\{\ell : S_\ell;T\}_{\ell \in L}} \qquad \textsc{TN-Identity}\ \frac{T \neq (U;T, \mu X.S, \oplus\{\ell : S_\ell\}_{\ell \in L})}{T \Downarrow T}$$

Fig. S-9. Type Normalisation

S-E.1 Type Normalisation

This section contains fig. S-9.

S-E.2 Implementation

S-E.2.1 Constraint Checking with FreeST. In the implementation, we use FreeST [Almeida et al. 2022, 2019, 2026] version 5.0 to perform equivalence checking. To this end, we generate little FreeST programs that typecheck if and only if the equivalence constraints hold.

For every equivalence constraint $T_1 \simeq T_2$, we generate the following FreeST program consisting of type definitions for T_1 and T_2 as well as an identity function mapping T_1 to T_2. For example, let $T_1 = \mu X.!\texttt{Int};!\texttt{String};X$ and $T_2 = \mu X.\texttt{Skip};!\texttt{Int};!\texttt{String};X$. For the constraint $T_1 \simeq T_2$ we generate the following FreeST program:

```
1
type X_T1 : 1S
type X_T1 = !Int; !String; X_T1

type T1 : 1S
type T1 = X_T1


type X_T2 : 1S
type X_T2 = Skip; !Int; !String; X_T2

type T2 : 1S
type T2 = X_T2

id : T1 → T2
id x = x
```

FreeST 5.0 requires specifying kinds for type definitions. Kind 1S says that the type needs to be a session type. If T1 and T2 are the translated types from the equivalence constraint, it is easy to see that this program only typechecks, if T1 and T2 are equivalent session types.

The main parts of the translation are:

- Translating inline recursion of the form $\mu\alpha.S$ to explicit recursion.
- Translating mobility and direction annotations on function and pair types.
- Translating effects.

To translate a μ-type to explicit recursion, we generate a fresh name and associate the name with the recursion unrolled and translation applied. We return the associated name, instead of the type. We encode mobility, direction, and effects in a branch type with labels indicating the state of the annotation. Thus, the translation function $[\![T]\!]$, defined in fig. S-10, takes a type T and returns a pair $\langle F, U\rangle$ consisting of a FreeST Type F and a list of explicit type definitions U.

S-E.2.2 Soundness and the Completeness of the translation.

Definition S-E.1. The equivalence relation $\simeq$ on CSTB types is the least congruence closed under the equivalence rules from Thiemann and Vasconcelos [2016] extended with rules from Figure S-11.

Let F be a FreeST session type and U be a mapping from variables to FreeST session types. In the following, we write $F[U]$ for the pair of F and U where $\text{freevars}(F) \subseteq \text{dom}(U)$. This notation includes unfolding of definitions in the following way:

$$\frac{(X \mapsto F) \in U}{X[U] \simeq F[U]}$$

$$
\begin{aligned}
[\![\texttt{Unit}]\!] &= \langle (), \emptyset \rangle \\
[\![\texttt{Skip}]\!] &= \langle \texttt{Skip}, [\,] \rangle \\
[\![\texttt{Close}]\!] &= \langle \texttt{Close}, [\,] \rangle \\
[\![\texttt{Wait}]\!] &= \langle \texttt{Wait}, [\,] \rangle \\
[\![T_1 \to^{m}_{d,\epsilon} T_2]\!] &= \langle (F_1 \to F_2) \otimes_d \&\{\ell : \texttt{Skip}\}_{\ell \in \text{dir} \cup \text{mob} \cup \text{eff}}, U_1 \cup U_2 \rangle \quad \text{where} \\
&\quad \langle F_1, U_1 \rangle = [\![T_1]\!] \\
&\quad \langle F_2, U_2 \rangle = [\![T_2]\!] \\
&\quad \text{dir} = \begin{cases} \{\text{left}\} & \text{if } d = \text{L} \\ \{\text{right}\} & \text{if } d = \text{R} \\ \{\text{lin}\} & \text{if } d = \mathbf{1} \\ \emptyset & \text{if } d = \omega \end{cases} \\
&\quad \text{mob} = \begin{cases} \{\text{mobile}\} & \text{if } m = \text{M} \\ \{\text{static}\} & \text{if } m = \text{S} \end{cases} \\
&\quad \text{eff} = \begin{cases} \{\text{impure}\} & \text{if } \epsilon = \text{I} \\ \{\} & \text{if } \epsilon = \text{I} \end{cases} \\
[\![T_1 \otimes_d T_2]\!] &= \langle (F_1 \times F_2) \otimes_d \&\{\ell : \texttt{Skip}\}_{\ell \in \text{dir}}, U_1 \cup U_2 \rangle \quad \text{where} \\
&\quad \langle F_1, U_1 \rangle = [\![T_1]\!] \\
&\quad \langle F_2, U_2 \rangle = [\![T_2]\!] \\
&\quad \text{dir} = \begin{cases} \{\text{left}\} & \text{if } d = \text{L} \\ \{\text{right}\} & \text{if } d = \text{R} \\ \{\text{lin}\} & \text{if } d = \mathbf{1} \\ \emptyset & \text{if } d = \omega \end{cases} \\
[\![\langle \ell : T_\ell \rangle_{\ell \in L}]\!] &= \langle \langle \ell : F_\ell \rangle_{\ell \in L}, \bigcup_{\ell \in L} U_\ell \rangle \quad \text{where} \\
&\quad \langle F_\ell, U_\ell \rangle = [\![T_\ell]\!] \quad \text{for every } \ell \in L \\
[\![!T]\!] &= \langle !F, U \rangle \quad \text{where} \\
&\quad \langle F, U \rangle = [\![T]\!] \\
[\![?T]\!] &= \langle ?F, U \rangle \quad \text{where} \\
&\quad \langle F, U \rangle = [\![T]\!] \\
[\![S_1; S_2]\!] &= \langle F_1; F_2, U_1 \cup U_2 \rangle \quad \text{where} \\
&\quad \langle F_1, U_1 \rangle = [\![S_1]\!] \\
&\quad \langle F_2, U_2 \rangle = [\![S_2]\!] \\
[\![\oplus\{\ell : S_\ell\}_{\ell \in L}]\!] &= \langle +\{\ell : F_\ell\}_{\ell \in L}, \bigcup_{\ell \in L} U_\ell \rangle \quad \text{where} \\
&\quad \langle F_\ell, U_\ell \rangle = [\![S_\ell]\!] \quad \text{for every } \ell \in L \\
[\![\&\{\ell : S_\ell\}_{\ell \in L}]\!] &= \langle \&\{\ell : F_\ell\}_{\ell \in L}, \bigcup_{\ell \in L} U_\ell \rangle \quad \text{where} \\
&\quad \langle F_\ell, U_\ell \rangle = [\![S_\ell]\!] \quad \text{for every } \ell \in L \\
[\![\texttt{Drop}]\!] &= \langle !\texttt{Drop}, \emptyset \rangle \\
[\![\texttt{Acq}]\!] &= \langle ?\texttt{Drop}, \emptyset \rangle \\
[\![\mu X.S]\!] &= \langle X', U \cup \{X' \mapsto U\} \rangle \quad \text{where } X' \text{ is fresh and} \\
&\quad \langle F, U \rangle = [\![S\,[X'/X]]\!] \\
[\![X]\!] &= \langle X, \emptyset \rangle
\end{aligned}
$$

Fig. S-10. Translation of types to FreeST types.

Lemma S-E.2 (Soundness of the translation). *If* $T_1 \simeq T_2$, *then* $F_1[U_1] \simeq F_2[U_2]$ *where* $\langle F_1, U_1 \rangle = [\![T_1]\!]$ *and* $\langle F_2, U_2 \rangle = [\![T_2]\!]$.

$$\frac{T_1 \simeq T_1' \qquad T_2 \simeq T_2'}{T_1 \to^m_{d,\epsilon} T_2 \simeq T_1' \to^m_{d,\epsilon} T_2'} \qquad \frac{T_1 \simeq T_1' \qquad T_2 \simeq T_2'}{T_1 \otimes_d T_2 \simeq T_1' \otimes_d T_2'} \qquad \frac{T_1 \simeq T_1' \qquad T_2 \simeq T_2'}{T_1 + T_2 \simeq T_1' + T_2'}$$

Fig. S-11. Modified rules for $\simeq$

Proof. We sketch a proof. The soundness proof requires two things: the translation of annotations in the arrow and product types preserves equivalence and the equivalence of the explicit recursion with the μ-binder syntax. For the former we can perform induction on the equivalence of $T_1 \simeq T_2$ and use equivalence and bisimulation lemmas from Thiemann and Vasconcelos [2016] for regular type and session type cases, respectively. The pairs we introduce for annotations can be handled trivially by the type equivalence rules. To show explicit recursion translation preserves equivalence, we can use the laws for μ-types from Thiemann and Vasconcelos [2016]. □

Lemma S-E.3 (Completeness of the translation). *Let* $\langle F_1, U_1 \rangle = [\![T_1]\!]$ *and* $\langle F_2, U_2 \rangle = [\![T_2]\!]$. *If* $F_1[U_1] \simeq F_2[U_2]$, *then* $T_1 \simeq T_2$.

Proof. Similar to S-E.2 but the opposite direction. □

B-ExpValueBlocked
$$\text{Blocked } \langle * \rangle$$

B-ExpConstBlocked
$$\frac{c \notin \{\texttt{new}, \texttt{fork}, \texttt{lsplit}, \texttt{rsplit}, \texttt{drop}, \texttt{discard}\}}{\text{Blocked } \mathcal{F}[c\, v]}$$

B-ParBlocked
$$\frac{\text{Blocked } P_1 \qquad \text{Blocked } P_2}{\text{Blocked } P_1 \parallel P_2}$$

B-NuBlocked
$$\frac{\{x, y\} \cap \mathsf{BC}(P) \neq \{x, y\} \qquad \text{Blocked } P}{\text{Blocked } (\nu[xB_1][yB_2])P}$$

B-NuBlockedAcq
$$\frac{x \notin \mathsf{AC}(P) \qquad B_i = \|xB \qquad \text{Blocked } P}{\text{Blocked } (\nu[B_1][B_2])P}$$

where

$$\begin{aligned}
&\mathsf{BC}(\mathcal{F}[\texttt{send}\,(v \otimes x)]) = \{x\} && \mathsf{BC}(\mathcal{F}[\texttt{recv}\,x]) = \{x\}\\
&\mathsf{BC}(\mathcal{F}[\texttt{select}\,k\,x]) = \{x\} && \mathsf{BC}(\mathcal{F}[\texttt{branch}\,x]) = \{x\}\\
&\mathsf{BC}(\mathcal{F}[\texttt{close}\,x]) = \{x\} && \mathsf{BC}(\mathcal{F}[\texttt{wait}\,x]) = \{x\}\\
&\mathsf{BC}(\mathcal{F}[\texttt{acquire}\,x]) = \emptyset && \mathsf{BC}(\mathcal{F}[\texttt{drop}\,x]) = \emptyset\\
&\mathsf{BC}(\mathcal{F}[\texttt{lsplit}\,x]) = \emptyset && \mathsf{BC}(\mathcal{F}[\texttt{rsplit}\,x]) = \emptyset\\
&\mathsf{BC}(\mathcal{F}[\texttt{fork}\,v]) = \emptyset && \mathsf{BC}(\mathcal{F}[\texttt{new}\,e]) = \emptyset\\
&\mathsf{BC}(\mathcal{F}[\texttt{discard}\,x]) = \emptyset
\end{aligned}$$

$$\begin{aligned}
&\mathsf{AC}(\mathcal{F}[\texttt{send}\,(v \otimes x)]) = \emptyset && \mathsf{AC}(\mathcal{F}[\texttt{recv}\,x]) = \emptyset\\
&\mathsf{AC}(\mathcal{F}[\texttt{select}\,k\,x]) = \emptyset && \mathsf{AC}(\mathcal{F}[\texttt{branch}\,x]) = \emptyset\\
&\mathsf{AC}(\mathcal{F}[\texttt{close}\,x]) = \emptyset && \mathsf{AC}(\mathcal{F}[\texttt{wait}\,x]) = \emptyset\\
&\mathsf{AC}(\mathcal{F}[\texttt{acquire}\,x]) = \{x\} && \mathsf{AC}(\mathcal{F}[\texttt{drop}\,x]) = \emptyset\\
&\mathsf{AC}(\mathcal{F}[\texttt{lsplit}\,x]) = \emptyset && \mathsf{AC}(\mathcal{F}[\texttt{rsplit}\,x]) = \emptyset\\
&\mathsf{AC}(\mathcal{F}[\texttt{fork}\,v]) = \emptyset && \mathsf{AC}(\mathcal{F}[\texttt{new}\,e]) = \emptyset\\
&\mathsf{AC}(\mathcal{F}[\texttt{discard}\,x]) = \emptyset
\end{aligned}$$

Fig. S-12. Blocked Processes (Blocked P)

S-F Metatheory

S-F.1 Type Safety for Declarative Typing

Lemma S-F.1 (Process progress). *If* $\Sigma \vdash e : \texttt{Unit} \mid \mathrm{I}$*, then* e *is a value, or* $e \to_e e'$ *for some* e'*, or* e *is blocked. If* e *is a value, then* $e = *$.

Proof. The trichotomy is expression progress; a stuck redex $\mathcal{E}[c\,v]$ is a blocked process. A value of type $\texttt{Unit}$ in a session context is $*$ by canonical forms. □

Theorem S-F.2 (Process progress, restated). *If* $\Sigma \vdash P$*, then* $P \equiv \langle * \rangle$*, or* Blocked P*, or* $P \to_p P'$ *for some* P'.

Proof. Induction on the derivation of $\Sigma \vdash P$; TP-Weaken follows from the induction hypothesis.

Case TP-Expr, $P = \langle e \rangle$, $\Sigma \vdash e : \texttt{Unit} \mid \mathrm{I}$. By theorem S-F.1:

- e a value: $e = *$ and $P = \langle * \rangle$.
- $e \to_e e'$: $P \to_p \langle e' \rangle$ by R-Exp.
- $e = \mathcal{E}[c\,v]$ blocked: if $c \in \{\texttt{new}, \texttt{fork}\}$ then P reduces by R-New, resp. R-Fork. Otherwise c acts on a channel $x \in \Sigma$, which no restriction binds, so no channel rule applies and Blocked P by B-ExpBlocked.

Case TP-Par, $P = P_1 \parallel P_2$. Apply the induction hypothesis to P_1 and to P_2.

- If $P_1 \to_p P_1'$, then $P \to_p P_1' \parallel P_2$ by R-Par; symmetrically, if $P_2 \to_p P_2'$, then $P \to_p P_1 \parallel P_2'$ by R-Par together with $P \equiv P_2 \parallel P_1$ (Par-Comm) and R-Struct.
- Otherwise, if $P_1 \equiv \langle * \rangle$, then $P \equiv P_2$ by Par-Unit, and the claim for P is the claim already established for P_2; symmetrically if $P_2 \equiv \langle * \rangle$.
- Otherwise P_1 and P_2 are both blocked, and Blocked P by B-ParBlocked.

Case TP-Res, $P = (\nu[B_1][B_2])Q$. By inversion Q is typed in Σ extended by B_1, B_2, again a session context. By the induction hypothesis on Q:

- $Q \to_p Q'$: then $P \to_p (\nu[B_1][B_2])Q'$ by R-Bind.
- $Q \equiv \langle * \rangle$: excluded. The endpoints of B_1, B_2 carry linear session types $S;\texttt{Close}$, $(S;\texttt{Wait})^{\perp}$, and $\vdash\!\!\rightsquigarrow$ requires each to be consumed by a process, so a body $\equiv \langle * \rangle$ is untypable.
- Q blocked. By the definition of Blocked , every process of Q is blocked and every restriction inside Q is blocked; the inner restrictions are closed by the induction hypothesis, so only the channel bound *here* remains. This restriction binds one channel with two dual endpoints: x, the head of B_1, governed by S, and y, the head of B_2, governed by $S^{\perp}$. (Earlier splits may have left further fragments in B_1, B_2; R-LSplit and R-RSplit act on such a fragment in place, wherever it sits in its group.) Exactly one of the following holds.
  (1) A process is blocked on a fragment of B_1 or B_2 with $\texttt{lsplit}$ or $\texttt{rsplit}$, or on x or y with $\texttt{drop}$ or $\texttt{discard}$, or on x or y with $\texttt{acquire}$ while the marker $\parallel$ is present. The matching rule R-LSplit, R-RSplit, R-Drop, R-Discard, resp. R-Acquire fires and P reduces.
  (2) Case (1) does not apply, and two distinct processes are blocked at communicating actions on the two ends of the bound channel: one on x and one on y. Since x and y are the two ends of a single channel their session types are dual, so the two actions form a matched pair, one of $(\texttt{send}, \texttt{recv})$, $(\texttt{select}\,k, \texttt{branch})$, $(\texttt{close}, \texttt{wait})$, and P reduces by R-Com, R-Choice, resp. R-Close.
  (3) Neither (1) nor (2) applies. Then at least one of x, y is not blocked at a communicating action, i.e. $x \notin \mathrm{BC}(Q)$ or $y \notin \mathrm{BC}(Q)$: the endpoint holds no process ready to communicate on it, so no synchronization with the other end is possible. This is the premise of B-NuBlocked, which, with Blocked Q, gives Blocked P.

□

S-F.2 Soundness and Completeness for Algorithmic Typing

References

Bernardo Almeida, Andreia Mordido, Peter Thiemann, and Vasco T. Vasconcelos. 2022. Polymorphic Lambda Calculus With Context-Free Session Types. *Inf. Comput.* 289, Part (2022), 104948. doi:10.1016/J.IC.2022.104948

Bernardo Almeida, Andreia Mordido, and Vasco T. Vasconcelos. 2019. FreeST: Context-free Session Types in a Functional Language. In *Proceedings Programming Language Approaches to Concurrency- and Communication-cEntric Software, PLACES@ETAPS 2019, Prague, Czech Republic, 7th April 2019 (EPTCS, Vol. 291)*, Francisco Martins and Dominic Orchard (Eds.). 12–23. doi:10.4204/EPTCS.291.2

Bernardo Almeida, Andreia Mordido, and Vasco T. Vasconcelos. 2026. FreeST, a Concurrent Programming Language With Context-Free Session Types. https://freest-lang.github.io.

Luca Padovani. 2019. Context-Free Session Type Inference. *ACM Trans. Program. Lang. Syst.* 41, 2 (2019), 9:1–9:37. doi:10.1145/3229062

Hannes Saffrich, Janek Spaderna, Peter Thiemann, and Vasco T. Vasconcelos. 2025. Borrowing from Session Types. *Proc. ACM Program. Lang.* 9, OOPSLA2 (2025), 3426–3453. doi:10.1145/3763173

Peter Thiemann and Vasco T. Vasconcelos. 2016. Context-free session types. In *Proceedings of the 21st ACM SIGPLAN International Conference on Functional Programming, ICFP 2016, Nara, Japan, September 18-22, 2016*. ACM, 462–475. doi:10.1145/2951913.2951926